FinSight AI — Complete Project Documentation & Technical Guide

Author: Pijush Pathak
Repository: https://github.com/PijushPathak07/Python_and_Gen-AI

---

1. Project Description

FinSight AI is a full-stack financial intelligence platform designed to help equity researchers, investors, and financial analysts make sense of lengthy corporate earnings call transcripts.

In the real world, when a public company releases its quarterly earnings, executives hold an hour-long call resulting in a 15-to-30-page transcript. Reading through pages of corporate jargon to find actual revenue numbers, expansion plans, or hidden risks takes hours. FinSight AI solves this problem by taking raw transcripts (either pasted as text, uploaded as a `.pdf` or `.txt` file, or processed in batches) and turning them into structured, actionable intelligence in seconds.

Beyond simple summarization, the platform acts as a complete research workstation. It scores companies across 6 core financial dimensions, detects when executives use "hedging" or evasive language, lets users ask follow-up questions backed by exact quotes from the transcript, compares multiple competitors in a sector heatmap, checks if management actually kept their promises from previous quarters, and connects directly to the U.S. government's live SEC EDGAR database to pull official `10-K`, `10-Q`, and `8-K` filings.

Complete Technology Stack

- **Backend Framework:** Python 3.10 / 3.11, Flask, Flask-CORS
- **AI & Natural Language Processing (NLP):** Google Gemini (`google-generativeai`) paired with a custom deterministic local NLP fallback engine (TF-IDF keyword overlap, regex sentence scoring, and financial lexicon matching)
- **Data Validation & Type Safety:** Pydantic v2 (`pydantic>=2.0.0`)
- **Database & Caching:** SQLite3 (`finsight.db`) with SHA-256 content-addressable hashing and full-text Retrieval-Augmented Generation (RAG) snippet search
- **Document & External Data Ingestion:** `pypdf` (for PDF extraction), `requests` (for live U.S. SEC EDGAR REST API integration), and `concurrent.futures.ThreadPoolExecutor` (for parallel batch processing)
- **Frontend UI:** HTML5, Tailwind CSS (Obsidian & Emerald dark-mode glassmorphic design), Vanilla JavaScript (`async/await`), and Chart.js (for 6-factor interactive radar charts)
- **Production Server & DevOps:** Gunicorn WSGI Server, Docker, Docker Compose, and GitHub Actions (CI/CD automated testing across Python 3.10 and 3.11)
- **API Documentation & Testing:** OpenAPI 3.0 (`/openapi.json`), Swagger UI (`/apidocs`), and Python `unittest` (`18` automated offline unit tests)

---

2. Files Structure

The project is organized into clean, modular files so that database logic, AI processing, data validation, and web routes remain completely separate and easy to maintain:

```
Python_and_Gen-AI/
│
├── .github/
│   └── workflows/
│       └── ci.yml                # GitHub Actions CI pipeline (runs 18 unit
tests on every push)
│
├── templates/
│   └── index.html                # Responsive 5-page dark-mode FinTech dashboard
& UI
│
├── app.py                        # Main Flask application, REST API routes, and
rate-limiter middleware
├── analyzer.py                   # Hybrid Gemini AI + Local NLP engine, Forensic
Scanner & Q&A logic
├── database.py                   # SQLite database setup, SHA-256 caching, RAG
search & peer benchmarking
├── schemas.py                    # Strict Pydantic v2 input/output validation
models
├── sec_connector.py              # Live connector to the U.S. SEC EDGAR API (CIK
& 10-K/10-Q lookup)
├── pdf_parser.py                 # Extracts and cleans text from uploaded .pdf
and .txt files
├── config.py                     # Loads environment variables (.env) and system
constants
├── test_api.py                   # 18 automated unit tests verifying every
endpoint and edge case
│
├── Dockerfile                    # Production container build instructions using
Python 3.11-slim
├── docker-compose.yml            # Multi-container orchestration with persistent
SQLite volume
├── .dockerignore                 # Excludes local virtual environments and temp
files from Docker
├── requirements.txt              # List of all Python libraries needed to run
the project
└── README.md                     # Quick-start documentation and architecture
overview
```

---

3. Project Overview

When an analyst or reviewer opens FinSight AI, the workflow happens in a clear, step-by-step pipeline:

1. **Multi-Format Input:** The user can paste an earnings transcript, click the "Load Demo Filing" button, upload a PDF/TXT document, or send up to 5 transcripts at once via the `/batch_analyze` endpoint.
2. **Input Sanitization & Validation:** Before any heavy processing happens, `schemas.py` checks the data using Pydantic v2. It trims extra whitespace, capitalizes stock tickers, ensures required fields aren't missing, and blocks oversized inputs that exceed the token limit.

3. **Zero-Cost SHA-256 Cache Check:** Next, `database.py` creates a unique digital fingerprint (SHA-256 hash) of the company name and transcript text. If that exact transcript was already analyzed in the past, the app immediately returns the saved report from SQLite in less than 5 milliseconds—using **zero API tokens**.
4. **Single-Shot Hybrid AI Extraction:** If the transcript is new, `analyzer.py` sends a single, carefully structured prompt to Google Gemini requesting a strict JSON response. This cuts API calls by 80% compared to asking separate questions for each category. If the user is running the app offline or without an API key, the system automatically switches to its built-in local heuristic NLP engine so the app never crashes.
5. **Interactive Exploration:** Once analyzed, the user can explore the report across five dedicated pages in the UI:
    - **Intelligence Studio:** View the 5-pillar summary, the 6-factor Radar Chart, the Forensic Red-Flag audit, and chat with the filing using Grounded Q&A.
    - **RAG & SEC EDGAR:** Search across all stored transcripts for specific keywords (like "*GPU*" or "*carbon tax*") or look up live U.S. SEC filings by stock ticker.
    - **Sector Peer Matrix:** Compare all analyzed companies in a ranked heatmap table.
    - **QoQ & Credibility Audit:** Compare two quarters side-by-side and check if management delivered on their earlier promises.
    - **System Architecture:** View live cache telemetry and open the interactive Swagger API documentation.

---

4. Project Objectives

The main goals behind building FinSight AI were:

- **Eliminate LLM Hallucinations in Finance:** Financial research requires strict accuracy. By forcing the Q&A engine (`POST /ask/<id>`) and Forensic Scanner (`GET /forensics/<id>`) to pull verbatim quotes directly from the source transcript, users can trust where every answer comes from.
- **Minimize API Latency and Cost:** Calling an LLM five separate times for five different summary categories is slow and expensive. Consolidating the extraction into a single structured JSON call—and backing it with a SHA-256 SQLite cache—makes the system fast and budget-friendly.
- **Ensure 100% Reliability (Offline & Online):** Many AI projects fail the moment an API key expires or a rate limit is hit. Designing a hybrid architecture with a local deterministic NLP fallback ensures the platform works seamlessly in any environment, including automated CI/CD test servers.
- **Go Beyond Basic Summarization:** Give analysts deeper tools they actually need, such as detecting when executives dodge tough questions (hedging analysis), tracking executive credibility across quarters, and benchmarking competitors side-by-side.
- **Follow Production Engineering Standards:** Enforce strict data schemas with Pydantic v2, protect endpoints with per-IP token-bucket rate limiting, document routes with OpenAPI 3.0, containerize with Docker, and verify everything with 18 unit tests.

---

5. Explanation About Code

Here is a plain-English breakdown of what each Python file does under the hood:

1. `config.py` (Configuration Manager)

This file uses `python-dotenv` to load settings from your `.env` file. It stores the `GOOGLE_API_KEY`, the Gemini model name (`gemini-2.5-flash`), the maximum allowed character/token length (`30,000`), and the path to the SQLite database (`finsight.db`). Keeping these in one file prevents hardcoded secrets in the main codebase.

2. `schemas.py` (Pydantic v2 Data Contracts)

Instead of writing messy `if/else` statements to check if user data is valid, this file defines strict classes using Pydantic v2:

- `TranscriptRequest`: Validates incoming transcripts, automatically strips leading/trailing spaces, converts tickers to uppercase, and raises a clear error if the text is empty or exceeds `30,000` characters.
- `AnalysisResponse` & `RadarScores`: Ensures that every AI report contains all 5 summary pillars and keeps all 6 radar scores strictly between `0` and `100`.
- `GroundedAnswerResponse`, `ForensicAuditResponse`, `SectorBenchmarkResponse`, `CredibilityTrackerResponse`, and `SECCompanyIntelligenceResponse`: Guarantee that every single API endpoint returns clean, predictable JSON.

3. `pdf_parser.py` (Document Ingestion & Cleaning)

When a user uploads a `.pdf` or `.txt` file, `extract_text_from_upload()` reads the raw bytes in memory without saving temporary junk files to your hard drive. It uses `pypdf` to extract text from every page of a PDF, and `normalize_transcript_text()` uses regular expressions (`re`) to clean up weird spacing, broken line breaks, and tab characters.

4. `database.py` (SQLite Persistence, Caching & RAG Search)

This file manages the local SQLite database (`finsight.db`):

- `compute_transcript_hash()`: Combines the lowercase company name and normalized transcript text to create a unique SHA-256 hash.
- `get_cached_report()` & `save_report()`: Checks if a hash already exists in the `reports` table and saves new analyses along with the raw transcript text.
- `search_reports_rag()`: Powers the `GET /search?q=<keyword>` endpoint. It searches all stored transcripts for a keyword, counts how many times the term appears, and extracts a 180-character context snippet around each match so the user can see the exact sentence where the keyword was used.
- `compute_sector_benchmark()`: Pulls multiple companies from the database, calculates a weighted Composite Score (`80%` Radar Average + `20%` Risk Safety), ranks them from `#1` to `#N`, and calculates sector-wide averages.
- `get_system_telemetry()`: Calculates total cached reports, sentiment breakdown (`Bullish`, `Neutral`, `Bearish`), average confidence, and estimated API tokens saved.

5. `analyzer.py` (The Core AI & Forensic NLP Brain)

This is the analytical engine of the project. It contains four major functions:

- `analyze_earnings_transcript()`: First calculates local NLP metrics (word count, reading time, and financial keyword hits like *EBITDA*, *margin*, *capex*). Then, if a valid Gemini API key is present, it sends a

single-shot prompt to Gemini and parses the JSON response. If no key is configured, it runs `build_local_heuristic_analysis()`, which scores sentences by keyword density to build the 5-pillar summary locally.

- `answer_transcript_question()`: Powers the "Chat with Filing" feature. It splits the transcript into individual sentences, filters out common stopwords, and scores each sentence based on keyword overlap with the user's question (giving a bonus to sentences that contain dollar signs or percentages). It returns a concise answer along with the top 3 verbatim sentences as citations.
- `run_forensic_red_flag_audit()`: Scans the transcript for "hedging" words (*potentially*, *uncertain*, *headwinds*, *challenging*) versus "certainty" words (*delivered*, *achieved*, *record*, *exceeded*). It also checks for accounting and regulatory risk patterns (like *Non-GAAP restructuring* or *litigation*) and calculates an overall **Transparency Score (0–100)**.
- `evaluate_guidance_credibility()`: Takes an earlier quarter's report and a newer quarter's report for the same company. It finds forward-looking promises made in the first report (sentences with words like *plan*, *expect*, *launch*, *target*) and checks the newer report to see if those goals were `Delivered`, `In Progress`, or `Missed / Pivoted`.

6. `sec_connector.py` (Live U.S. SEC EDGAR Integration)

Connects directly to the U.S. Securities and Exchange Commission's public data servers (`www.sec.gov` and `data.sec.gov`) with zero API keys required.

- `resolve_cik_from_sec()`: Downloads the SEC's official ticker registry to convert a stock symbol like `AAPL` or `NVDA` into its official 10-digit Central Index Key (`CIK`).
- `fetch_live_sec_edgar_profile()`: Uses that 10-digit CIK to pull the company's official legal name, SIC industry classification, stock exchange, and direct links to its latest `10-K` (annual), `10-Q` (quarterly), and `8-K` (current event) filings.

7. `app.py` (Flask Server, Middleware & OpenAPI Docs)

Ties all modules together into a clean REST API:

- **Rate-Limiting & Latency Middleware (`before_request` & `after_request`):** Tracks requests per IP address using an in-memory token bucket (`60 requests per 60 seconds`). If someone spams the server, it returns `429 Too Many Requests`. It also measures how many milliseconds each request takes and attaches `X-Process-Time-Ms`, `X-RateLimit-Limit`, and `X-RateLimit-Remaining` headers to every response.
- **Concurrent Batch Route (`POST /batch_analyze`):** Uses Python's `ThreadPoolExecutor` to analyze up to 5 transcripts simultaneously in parallel threads.
- **Export Route (`GET /export/<id>`):** Dynamically generates downloadable Executive Briefs in Markdown (`.md`) or spreadsheet-ready CSV (`.csv`) formats.
- **Built-In Swagger UI (`/apidocs` & `/openapi.json`):** Serves interactive API documentation right in the browser without needing external documentation libraries that might break across Python versions.

8. `test_api.py` (18 Automated Unit Tests)

Creates an isolated temporary SQLite database and uses `unittest.mock` to test all 18 workflows offline in under `0.3` seconds. It tests health checks, UI rendering, Pydantic validation errors, SHA-256 cache hits, RAG search, Swagger docs, concurrent batch analysis, Markdown/CSV exports, telemetry metrics, grounded Q&A

citations, forensic red-flag audits, rate-limiting `429` blocks, sector peer benchmarking, multi-quarter credibility audits, and live SEC EDGAR parsing.

---

6. Setup Guide

Follow these steps to run FinSight AI on your local computer or via Docker.

Prerequisites

- **Python 3.10 or higher** installed on your system
- **Git** installed
- *(Optional)* A free **Google Gemini API Key** from Google AI Studio (Note: The app works 100% offline even without a key thanks to its built-in local NLP engine!)

Method 1: Standard Local Setup (Windows / macOS / Linux)

Step 1: Clone the repository and open the folder

```
git clone https://github.com/PijushPathak07/Python_and_Gen-AI.git
cd Python_and_Gen-AI
```

Step 2: Create and activate a virtual environment

```
# On Windows (PowerShell):
python -m venv venv
.\venv\Scripts\Activate.ps1

# On macOS / Linux:
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install the required dependencies

```
pip install -r requirements.txt
```

Step 4: Configure environment variables (Optional) Create a file named `.env` in the root folder and add your Gemini API key:

```
GOOGLE_API_KEY=your_actual_gemini_api_key_here
GEMINI_MODEL_NAME=gemini-2.5-flash
MAX_TOKENS=30000
DATABASE_PATH=finsight.db
```

Step 5: Run the 18 automated unit tests to verify your setup

```
python -m unittest test_api.py -v
```

(You should see `Ran 18 tests ... OK`)

Step 6: Start the application

```
python app.py
```

- Open `http://127.0.0.1:5000` in your browser to use the FinSight AI Dashboard.
- Open `http://127.0.0.1:5000/apidocs` to test the endpoints in Swagger UI.

Method 2: One-Command Docker Setup

If you have Docker Desktop installed, you can spin up the production Gunicorn server (with 4 worker processes and persistent SQLite storage) using a single command:

```
docker-compose up --build -d
```

The service will be live at `http://localhost:5000`.

---

7. Future Reference (Roadmap & Scalability)

While FinSight AI is already a complete, production-ready platform, here is how the architecture can be scaled in the future as user traffic and data volume grow:

1. **Vector Database Upgrade (FAISS / pgvector):** Currently, the RAG search and Grounded Q&A use deterministic keyword-overlap and TF-IDF sentence scoring in SQLite. Integrating dense vector embeddings (such as Google's `text-embedding-004` stored in PostgreSQL with `pgvector` or FAISS) will allow semantic similarity search across thousands of 100-page annual reports.
2. **Automated SEC Filing Text Ingestion:** Right now, the SEC EDGAR connector fetches official filing metadata and direct document links. A natural next step is adding an HTML/XBRL parser that automatically downloads the full text of a `10-K` or `10-Q` filing directly into the analyzer with one click.
3. **Audio Earnings Call Transcription (Whisper / Gemini Multimodal):** Allow analysts to upload raw `.mp3` or `.wav` recordings of live earnings calls so the system can transcribe the audio and detect vocal tone hesitation alongside textual hedging.
4. **Distributed Caching & Task Queues (Redis + Celery):** When scaling from 5 concurrent transcripts to hundreds of institutional users, replacing the in-memory rate limiter and SQLite cache with **Redis**, and moving batch processing to **Celery** background workers, will allow horizontal scaling across multiple cloud servers.

5. **Role-Based Access Control (JWT Authentication):** Add user accounts and API key management so investment teams can maintain private workspaces, custom watchlists, and shared annotation notes.